IMPLEMENTACIÓN DEL MODELO FÍSICO

Red Social Estudiantil Pascualina

Tarea 3 · Bases de Datos I

Integrante

Jonathan Andrés Serna Velasco

Tutora: Oralia Cortes Grajales

Repositorio Git: bd1-redsocial-jonathan-andres-serna-velasco

Octubre de 2026

Contenido

- 1. Introducción y alcance
- 2. Identificación del SGBD y justificación de su elección
- 3. Del modelo lógico al modelo físico: cómo fue transformado
- 4. Descripción de cada tabla implementada (24 tablas)
- 5. Implementación de las relaciones entre tablas (36 llaves foráneas)
- 6. El script DDL: estructura y verificación
- 7. Conclusiones
- Anexo. Diagrama del modelo físico

1. Introducción y alcance

Este informe documenta la implementación, en un sistema gestor de base de datos real, del modelo físico de la red social estudiantil Pascualina. El punto de partida es el modelo lógico de 24 tablas elaborado en la Tarea 2 (derivado a su vez del modelo conceptual aprobado en la Tarea 1). La plataforma permite crear perfiles, conectar con compañeros, publicar actualizaciones, crear y unirse a grupos, y programar eventos.

El trabajo se organizó en un repositorio Git y se materializa en tres entregables: el **script DDL** (`ddl_red_social.sql`), este **informe técnico** y un **video de sustentación**. Además se incluye un script de pruebas (`pruebas_red_social.sql`) que demuestra que las restricciones definidas funcionan.

2. Identificación del SGBD y justificación de su elección

Se eligió **PostgreSQL 16**, administrado con **pgAdmin4**. Las opciones consideradas fueron:

SGBD	Ventajas para este proyecto	Desventajas / limitaciones
PostgreSQL (elegido)	Código abierto y gratuito. Cumple de forma muy estricta el estándar SQL: llaves foráneas, CHECK, UNIQUE y acciones referenciales (RESTRICT, CASCADE, SET NULL) se hacen cumplir siempre. Columnas IDENTITY, esquemas (`red_social`), tipos SMALLINT/INTEGER/BIGINT, TIMESTAMP, BOOLEAN y TEXT nativos. Documentación y material de consulta del curso (Canepa; Hsu y Obe) están orientados a PostgreSQL. pgAdmin4 permite diseñar y probar visualmente.	Administración y configuración inicial algo más extensas que en MySQL. No crea índices automáticamente sobre las llaves foráneas (hubo que declararlos explícitamente).
MySQL 8 / MariaDB	Muy difundido, instalación sencilla, MySQL Workbench como herramienta gráfica. Con el motor InnoDB soporta llaves foráneas y, desde la versión 8.0.16, restricciones CHECK.	Menor apego al estándar SQL y menos funciones avanzadas. El comportamiento de algunas restricciones depende del motor de almacenamiento y de la versión; un CHECK en versiones antiguas se aceptaba pero no se aplicaba.
SQL Server (Express)	Excelente herramienta gráfica (SSMS) y buen rendimiento.	Software propietario; la edición gratuita tiene límites de tamaño por base de datos. Exige más recursos y está menos alineado con los materiales del curso.
SQLite	Sin servidor, ideal para prototipos y aplicaciones locales.	Una red social es multiusuario y concurrente; SQLite admite un solo escritor a la vez y exige activar las llaves foráneas manualmente (PRAGMA).

Conclusión de la elección: PostgreSQL ofrece la mejor combinación de integridad referencial estricta, tipos de datos precisos y herramientas gratuitas, lo cual es lo central en un ejercicio cuyo objetivo es aplicar con rigor las reglas del lenguaje de definición de datos.

3. Del modelo lógico al modelo físico: cómo fue transformado

El modelo lógico de la Tarea 2 define tablas, atributos, claves y relaciones de forma independiente del SGBD. El modelo físico concreta esas definiciones para PostgreSQL. Las transformaciones aplicadas fueron:

Aspecto	Modelo lógico (Tarea 2)	Modelo físico (Tarea 3)
Nombres de tablas	MAYÚSCULAS, singular (USUARIO_GRUPO)	minúsculas, singular, snake_case, sin tildes (`usuario_grupo`). Se conserva la correspondencia 1 a 1 con el modelo lógico.
Organización	Sin esquema	Todas las tablas en el esquema `red_social`.
Llaves primarias simples	INT(11) autoincremental	`INTEGER GENERATED ALWAYS AS IDENTITY` (estándar SQL; evita el uso de SERIAL). BIGINT en las tablas de mayor crecimiento: publicacion, comentario, adjunto y mensaje.
Fechas	DATETIME	`TIMESTAMP` con `DEFAULT CURRENT_TIMESTAMP` en las fechas de registro.
Texto	VARCHAR(n), TEXT	Se mantienen las longitudes de la Tarea 2; `TEXT` solo para el cuerpo de publicacion.
Valores permitidos	Descritos en texto (p. ej. 'activa, finalizada')	Restricciones `CHECK ... IN (...)` para tipo_publicacion, tipo_grupo, modalidad, estado, etc.
Nulabilidad	Indicada solo en algunos campos	Todo campo es `NOT NULL` salvo los opcionales declarados en el modelo lógico.
Relaciones	Cardinalidad e FK en lenguaje descriptivo	Cada relación es una `FOREIGN KEY` con `ON DELETE` y `ON UPDATE` explícitos (sección 5).
Especialización	Subtablas estudiante y docente con PK = FK	Igual; implementada con PK y FK sobre `id_usuario`.
Rendimiento	No considerado	Índices sobre las llaves foráneas que no encabezan una PK compuesta (24 índices).

Ajustes realizados al modelo lógico

Al pasar a físico se detectaron puntos que convenía corregir o precisar. Se documentan para mantener la trazabilidad:

- **docente_asignatura:** en la Tarea 2 la PK era (id_docente, id_asignatura), pero el campo periodo_academico se justificaba para conservar el historial. Con esa PK un docente no podía orientar la misma asignatura en dos periodos. En el modelo físico la PK es **(id_docente, id_asignatura, periodo_academico)**.
- **Unicidad de programa_academico.nombre_programa:** se añadió UNIQUE para evitar programas duplicados en el catálogo.
- **Reglas de integridad adicionales (CHECK):** nadie puede seguirse a sí mismo (sigue), ser su propio mentor (mentoria) ni enviarse mensajes a sí mismo (mensaje); la hora de fin de un evento debe ser posterior a la de inicio; el semestre debe estar entre 1 y 14; el periodo académico sigue el formato AAAA-1 o AAAA-2.
- **Valores por defecto:** fechas automáticas, rol_en_grupo = 'miembro', estado_asistencia = 'interesado', estado de mentoría = 'activa', leido = FALSE.
- **Campos opcionales:** descripcion de grupo y de evento se dejaron NULL porque el modelo lógico no los declaraba obligatorios.

4. Descripción de cada tabla implementada

Para cada tabla se indica su propósito, el listado de campos con su tipo y tamaño, la justificación de los tipos y la identificación y justificación de la clave primaria. Las justificaciones de la elección de tipos de datos (por ejemplo VARCHAR frente a TEXT, o INTEGER frente a BIGINT) se detallan en la última columna.

4.1 programa_academico

Catálogo institucional de programas académicos (carreras). Grupo: Estructura académica.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_programa	INTEGER	PK NOT NULL IDENTITY	Identificador del programa.	INTEGER: el catálogo de programas es pequeño (decenas); BIGINT desperdiciaría espacio.
nombre_programa	VARCHAR(100)	UNIQUE NOT NULL	Nombre del programa (p. ej. Ingeniería de Sistemas).	VARCHAR(100) cubre nombres largos; se limita para validar la entrada.
facultad	VARCHAR(100)	NOT NULL	Facultad a la que pertenece.	Texto corto de longitud variable.

Clave primaria: pk_programa_academico (id_programa). Clave sustituta autoincremental: estable, sin significado de negocio y eficiente en JOIN.

4.2 asignatura

Catálogo institucional de asignaturas o materias. Grupo: Estructura académica.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_asignatura	INTEGER	PK NOT NULL IDENTITY	Identificador de la asignatura.	INTEGER: catálogo pequeño.
nombre_asignatura	VARCHAR(100)	NOT NULL	Nombre de la materia.	VARCHAR(100): nombres de materias universitarias.
codigo_asignatura	VARCHAR(20)	UNIQUE NOT NULL	Código institucional oficial.	VARCHAR y no INTEGER: los códigos pueden mezclar letras y ceros a la izquierda.

Clave primaria: pk_asignatura (id_asignatura). Clave sustituta; el código institucional queda como UNIQUE (identificador de negocio).

4.3 usuario

Datos comunes a todo miembro de la comunidad (estudiante o docente). Grupo: Identidad y perfil.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_usuario	INTEGER	PK NOT NULL IDENTITY	Identificador del usuario.	INTEGER (hasta ~2.147 millones) alcanza para una comunidad universitaria; ocupa la mitad que BIGINT en cada FK.
nombre_completo	VARCHAR(150)	NOT NULL	Nombres y apellidos.	VARCHAR(150) admite nombres compuestos largos.
nombre_usuario	VARCHAR(50)	UNIQUE NOT NULL	Alias público único.	VARCHAR(50), único: identifica el perfil en la plataforma.
correo_institucional	VARCHAR(100)	UNIQUE NOT NULL CHECK (correo_institucional LIKE '%_@_%._%')	Correo de registro y acceso.	VARCHAR(100), único: identifica la cuenta.
contrasena_hash	VARCHAR(255)	NOT NULL	Hash de la contraseña.	255 caracteres cubren hashes bcrypt/Argon2; nunca texto plano.
foto_perfil_url	VARCHAR(255)	NULL	Enlace a la foto de perfil.	Solo la URL, no el binario; opcional.
biografia	VARCHAR(500)	NULL	Presentación personal breve.	VARCHAR(500) y no TEXT: se quiere acotar la longitud; opcional.
fecha_registro	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Fecha y hora de creación de la cuenta.	TIMESTAMP con valor por defecto automático.

Clave primaria: pk_usuario (id_usuario). Clave sustituta autoincremental; el correo y el alias son UNIQUE pero pueden cambiar, por eso no son PK.

4.4 estudiante

Subtipo de usuario con los atributos exclusivos del estudiante (relación 1:1). Grupo: Identidad y perfil.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_usuario	INTEGER	PK FK → usuario NOT NULL	Usuario que es estudiante.	Mismo tipo que usuario.id_usuario (PK y FK a la vez).
codigo_estudiantil	VARCHAR(20)	UNIQUE NOT NULL	Código institucional del estudiante.	VARCHAR: puede contener letras o ceros a la izquierda.
id_programa_academico	INTEGER	FK → programa_academico NOT NULL	Programa al que pertenece.	Mismo tipo que programa_academico.id_programa.
semestre_actual	SMALLINT	NOT NULL CHECK (semestre_actual BETWEEN 1 AND 14)	Semestre que cursa.	SMALLINT en lugar de INTEGER: el valor nunca supera unas pocas decenas.

Clave primaria: pk_estudiante (id_usuario). La PK es también FK a usuario: implementa la especialización 1:1 sin duplicar datos generales.

4.5 docente

Subtipo de usuario con los atributos exclusivos del docente (relación 1:1). Grupo: Identidad y perfil.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_usuario	INTEGER	PK FK → usuario NOT NULL	Usuario que es docente.	Mismo tipo que usuario.id_usuario.
codigo_docente	VARCHAR(20)	UNIQUE NOT NULL	Código institucional del docente.	VARCHAR(20), único.
titulo_academico	VARCHAR(100)	NOT NULL	Máximo título (Magíster, Doctor…).	Texto libre corto: la redacción varía por facultad.
departamento	VARCHAR(100)	NOT NULL	Departamento o unidad de adscripción.	Texto corto.

Clave primaria: pk_docente (id_usuario). PK y FK a la vez, igual que estudiante.

4.6 habilidad

Catálogo de habilidades (técnicas, deportivas, blandas, artísticas). Grupo: Identidad y perfil.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_habilidad	INTEGER	PK NOT NULL IDENTITY	Identificador de la habilidad.	INTEGER: catálogo.
nombre_habilidad	VARCHAR(100)	UNIQUE NOT NULL	Nombre (Python, Oratoria, Fútbol…).	Único para evitar duplicados conceptuales.
categoria	VARCHAR(50)	NOT NULL	Clasificación general.	Texto corto.

Clave primaria: pk_habilidad (id_habilidad). Clave sustituta.

4.7 interes

Catálogo de intereses o aficiones. Grupo: Identidad y perfil.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_interes	INTEGER	PK NOT NULL IDENTITY	Identificador del interés.	INTEGER: catálogo.
nombre_interes	VARCHAR(100)	UNIQUE NOT NULL	Nombre del interés.	Único para evitar duplicados.

Clave primaria: pk_interes (id_interes). Clave sustituta.

4.8 usuario_habilidad

Resuelve la relación N:M POSEE entre usuario y habilidad. Grupo: Identidad y perfil.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_usuario	INTEGER	PK FK → usuario NOT NULL	Usuario que declara la habilidad.	Mismo tipo que la PK referenciada.
id_habilidad	INTEGER	PK FK → habilidad NOT NULL	Habilidad declarada.	Mismo tipo que la PK referenciada.
nivel_competencia	VARCHAR(20)	NOT NULL CHECK (nivel_competencia IN ('basico', 'intermedio', 'avanzado'))	Nivel declarado.	Valores acotados con CHECK.

Clave primaria: pk_usuario_habilidad (id_usuario, id_habilidad). PK compuesta: impide que un usuario repita la misma habilidad.

4.9 usuario_interes

Resuelve la relación N:M TIENE_INTERES entre usuario e interés. Grupo: Identidad y perfil.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_usuario	INTEGER	PK FK → usuario NOT NULL	Usuario que declara el interés.	Mismo tipo que la PK referenciada.
id_interes	INTEGER	PK FK → interes NOT NULL	Interés declarado.	Mismo tipo que la PK referenciada.

Clave primaria: pk_usuario_interes (id_usuario, id_interes). PK compuesta: sin intereses repetidos por usuario.

4.10 sigue

Relación reflexiva N:M de seguidores entre usuarios. Grupo: Identidad y perfil.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_usuario_seguidor	INTEGER	PK FK → usuario NOT NULL CHECK (id_usuario_seguidor <> id_usuario_seguido)	Usuario que sigue.	Mismo tipo que usuario.id_usuario.
id_usuario_seguido	INTEGER	PK FK → usuario NOT NULL CHECK (id_usuario_seguidor <> id_usuario_seguido)	Usuario seguido.	Mismo tipo que usuario.id_usuario.
fecha_inicio_seguimiento	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Cuándo empezó el seguimiento.	TIMESTAMP automático.

Clave primaria: pk_sigue (id_usuario_seguidor, id_usuario_seguido). PK compuesta (seguidor, seguido): una sola relación por pareja. El CHECK evita seguirse a sí mismo.

4.11 programa_asignatura

Resuelve la relación N:M OFRECE entre programa académico y asignatura. Grupo: Estructura académica.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_programa	INTEGER	PK FK → programa_academico NOT NULL	Programa que ofrece la asignatura.	Mismo tipo que la PK referenciada.
id_asignatura	INTEGER	PK FK → asignatura NOT NULL	Asignatura ofrecida.	Mismo tipo que la PK referenciada.
semestre_sugerido	SMALLINT	NOT NULL CHECK (semestre_sugerido BETWEEN 1 AND 14)	Semestre recomendado para cursarla.	SMALLINT: número pequeño.

Clave primaria: pk_programa_asignatura (id_programa, id_asignatura). PK compuesta (programa, asignatura).

4.12 docente_asignatura

Resuelve la relación N:M ORIENTA entre docente y asignatura, por periodo académico. Grupo: Estructura académica.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_docente	INTEGER	PK FK → docente NOT NULL	Docente que orienta la asignatura.	Mismo tipo que docente.id_usuario.
id_asignatura	INTEGER	PK FK → asignatura NOT NULL	Asignatura orientada.	Mismo tipo que la PK referenciada.
periodo_academico	VARCHAR(20)	PK NOT NULL CHECK (periodo_academico ~ '^[0-9]{4}-[12]$')	Periodo en que se dicta (p. ej. 2026-2).	VARCHAR y no DATE: es una etiqueta (año-semestre); validada con CHECK.

Clave primaria: pk_docente_asignatura (id_docente, id_asignatura, periodo_academico). AJUSTE respecto a la Tarea 2: el periodo entra en la PK para que un docente pueda orientar la misma asignatura en periodos distintos (historial), que era el propósito declarado de ese campo.

4.13 mentoria

Relación reflexiva N:M entre estudiantes (mentor y aprendiz). Grupo: Estructura académica.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_usuario_mentor	INTEGER	PK FK → estudiante NOT NULL CHECK (id_usuario_mentor <> id_usuario_aprendiz)	Estudiante mentor.	Mismo tipo que estudiante.id_usuario.
id_usuario_aprendiz	INTEGER	PK FK → estudiante NOT NULL CHECK (id_usuario_mentor <> id_usuario_aprendiz)	Estudiante aprendiz.	Mismo tipo que estudiante.id_usuario.
id_asignatura	INTEGER	FK → asignatura NULL	Asignatura de la mentoría (opcional).	Nula si la mentoría es general.
fecha_inicio	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Inicio de la mentoría.	TIMESTAMP automático.
estado	VARCHAR(20)	NOT NULL DEFAULT 'activa' CHECK (estado IN ('activa', 'finalizada'))	Estado de la mentoría.	Valores acotados con CHECK.

Clave primaria: pk_mentoria (id_usuario_mentor, id_usuario_aprendiz). PK compuesta (mentor, aprendiz). El CHECK impide que alguien sea su propio mentor.

4.14 grupo

Grupo de estudio, equipo de hackatón o club de interés. Grupo: Grupos y eventos.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_grupo	INTEGER	PK NOT NULL IDENTITY	Identificador del grupo.	INTEGER: la cantidad de grupos es moderada.
nombre_grupo	VARCHAR(100)	NOT NULL	Nombre del grupo.	VARCHAR(100).
descripcion	VARCHAR(500)	NULL	Propósito del grupo.	VARCHAR(500) acotado; opcional.
tipo_grupo	VARCHAR(30)	NOT NULL CHECK (tipo_grupo IN ('materia', 'hackaton', 'club_interes'))	materia, hackaton o club_interes.	Valores acotados con CHECK.
id_usuario_creador	INTEGER	FK → usuario NOT NULL	Usuario que creó el grupo.	Mismo tipo que usuario.id_usuario.
id_asignatura	INTEGER	FK → asignatura NULL	Asignatura en la que se enfoca (opcional).	Nula para clubes sin materia.
fecha_creacion	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Fecha de creación.	TIMESTAMP automático.

Clave primaria: pk_grupo (id_grupo). Clave sustituta.

4.15 usuario_grupo

Resuelve la relación N:M PERTENECE entre usuario y grupo. Grupo: Grupos y eventos.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_usuario	INTEGER	PK FK → usuario NOT NULL	Usuario miembro.	Mismo tipo que la PK referenciada.
id_grupo	INTEGER	PK FK → grupo NOT NULL	Grupo al que pertenece.	Mismo tipo que la PK referenciada.
rol_en_grupo	VARCHAR(20)	NOT NULL DEFAULT 'miembro' CHECK (rol_en_grupo IN ('miembro', 'administrador'))	miembro o administrador.	Valores acotados con CHECK.
fecha_ingreso	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Cuándo se unió.	TIMESTAMP automático.

Clave primaria: pk_usuario_grupo (id_usuario, id_grupo). PK compuesta: sin membresías duplicadas.

4.16 evento

Reunión de estudio, taller o actividad social programada. Grupo: Grupos y eventos.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_evento	INTEGER	PK NOT NULL IDENTITY	Identificador del evento.	INTEGER.
nombre_evento	VARCHAR(100)	NOT NULL	Nombre del evento.	VARCHAR(100).
descripcion	VARCHAR(500)	NULL	Detalle del evento.	VARCHAR(500) acotado; opcional.
fecha_hora_inicio	TIMESTAMP	NOT NULL CHECK (fecha_hora_fin > fecha_hora_inicio)	Inicio.	TIMESTAMP: se necesita fecha y hora.
fecha_hora_fin	TIMESTAMP	NOT NULL CHECK (fecha_hora_fin > fecha_hora_inicio)	Fin.	TIMESTAMP; el CHECK exige que sea posterior al inicio.
modalidad	VARCHAR(20)	NOT NULL CHECK (modalidad IN ('presencial', 'virtual'))	presencial o virtual.	Valores acotados con CHECK.
ubicacion_o_enlace	VARCHAR(255)	NOT NULL	Dirección física o enlace de videollamada.	VARCHAR(255): cabe una URL típica.
id_usuario_organizador	INTEGER	FK → usuario NOT NULL	Organizador del evento.	Mismo tipo que usuario.id_usuario.
id_grupo	INTEGER	FK → grupo NULL	Grupo que lo programa (opcional).	Nulo si es un evento individual.

Clave primaria: pk_evento (id_evento). Clave sustituta.

4.17 usuario_evento

Resuelve la relación N:M ASISTE entre usuario y evento. Grupo: Grupos y eventos.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_usuario	INTEGER	PK FK → usuario NOT NULL	Usuario que asiste.	Mismo tipo que la PK referenciada.
id_evento	INTEGER	PK FK → evento NOT NULL	Evento al que asiste.	Mismo tipo que la PK referenciada.
estado_asistencia	VARCHAR(20)	NOT NULL DEFAULT 'interesado' CHECK (estado_asistencia IN ('interesado', 'confirmado', 'asistio'))	interesado, confirmado o asistio.	Valores acotados con CHECK.

Clave primaria: pk_usuario_evento (id_usuario, id_evento). PK compuesta: un registro por usuario y evento.

4.18 publicacion

Publicación en el muro: pregunta, recurso, noticia o meme. Grupo: Contenido social.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_publicacion	BIGINT	PK NOT NULL IDENTITY	Identificador de la publicación.	BIGINT en lugar de INTEGER: es la tabla de mayor crecimiento junto a comentario y mensaje.
id_usuario_autor	INTEGER	FK → usuario NOT NULL	Autor.	Mismo tipo que usuario.id_usuario.
id_grupo	INTEGER	FK → grupo NULL	Grupo donde se publicó (opcional).	Nulo si va al muro general.
tipo_publicacion	VARCHAR(20)	NOT NULL CHECK (tipo_publicacion IN ('pregunta', 'recurso', 'noticia', 'meme'))	pregunta, recurso, noticia o meme.	Valores acotados con CHECK.
contenido	TEXT	NOT NULL	Cuerpo de la publicación.	TEXT y no VARCHAR: no se quiere imponer un límite corto al contenido.
fecha_publicacion	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Fecha y hora.	TIMESTAMP automático.

Clave primaria: pk_publicacion (id_publicacion). Clave sustituta BIGINT.

4.19 etiqueta

Catálogo de etiquetas temáticas para publicaciones. Grupo: Contenido social.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_etiqueta	INTEGER	PK NOT NULL IDENTITY	Identificador de la etiqueta.	INTEGER: catálogo.
nombre_etiqueta	VARCHAR(50)	UNIQUE NOT NULL	Texto de la etiqueta (p. ej. Calculo).	VARCHAR(50): convención de hashtags cortos; único.

Clave primaria: pk_etiqueta (id_etiqueta). Clave sustituta.

4.20 publicacion_etiqueta

Resuelve la relación N:M SE_ETIQUETA_CON entre publicación y etiqueta. Grupo: Contenido social.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_publicacion	BIGINT	PK FK → publicacion NOT NULL	Publicación etiquetada.	BIGINT: igual que publicacion.id_publicacion.
id_etiqueta	INTEGER	PK FK → etiqueta NOT NULL	Etiqueta asignada.	Mismo tipo que la PK referenciada.

Clave primaria: pk_publicacion_etiqueta (id_publicacion, id_etiqueta). PK compuesta: sin etiquetas repetidas en una publicación.

4.21 adjunto

Archivo adjunto (imagen, video, documento o enlace) de una publicación. Entidad débil. Grupo: Contenido social.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_adjunto	BIGINT	PK NOT NULL IDENTITY	Identificador del adjunto.	BIGINT: pueden existir varios adjuntos por publicación.
id_publicacion	BIGINT	FK → publicacion NOT NULL	Publicación a la que pertenece.	BIGINT: igual que la PK referenciada.
url_archivo	VARCHAR(255)	NOT NULL	Ruta o enlace del archivo.	Solo la referencia, no el binario.
tipo_archivo	VARCHAR(20)	NOT NULL CHECK (tipo_archivo IN ('imagen', 'video', 'documento', 'enlace'))	imagen, video, documento o enlace.	Valores acotados con CHECK.

Clave primaria: pk_adjunto (id_adjunto). Clave sustituta propia para poder referenciar y eliminar cada adjunto.

4.22 comentario

Comentario sobre una publicación, con respuestas anidadas. Entidad débil. Grupo: Contenido social.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_comentario	BIGINT	PK NOT NULL IDENTITY	Identificador del comentario.	BIGINT: alto volumen.
id_publicacion	BIGINT	FK → publicacion NOT NULL	Publicación comentada.	BIGINT: igual que la PK referenciada.
id_usuario_autor	INTEGER	FK → usuario NOT NULL	Autor del comentario.	Mismo tipo que usuario.id_usuario.
id_comentario_padre	BIGINT	FK → comentario NULL	Comentario al que responde (opcional).	Nulo en comentarios de primer nivel.
contenido	VARCHAR(500)	NOT NULL	Texto del comentario.	VARCHAR(500) y no TEXT: los comentarios deben ser más cortos que las publicaciones.
fecha_comentario	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Fecha y hora.	TIMESTAMP automático.

Clave primaria: pk_comentario (id_comentario). Clave sustituta propia; necesaria para la autorreferencia de respuestas.

4.23 reaccion

Resuelve la relación N:M REACCIONA entre usuario y publicación. Grupo: Contenido social.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_usuario	INTEGER	PK FK → usuario NOT NULL	Usuario que reacciona.	Mismo tipo que usuario.id_usuario.
id_publicacion	BIGINT	PK FK → publicacion NOT NULL	Publicación reaccionada.	BIGINT: igual que la PK referenciada.
tipo_reaccion	VARCHAR(20)	NOT NULL CHECK (tipo_reaccion IN ('me_gusta', 'util', 'celebrar', 'divertido'))	me_gusta, util, celebrar o divertido.	Valores acotados con CHECK.
fecha_reaccion	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Cuándo reaccionó.	TIMESTAMP automático.

Clave primaria: pk_reaccion (id_usuario, id_publicacion). PK compuesta: una sola reacción por usuario y publicación.

4.24 mensaje

Mensaje directo entre dos usuarios. Grupo: Contenido social.

Campo	Tipo y tamaño	Restricciones	Descripción	Justificación del tipo
id_mensaje	BIGINT	PK NOT NULL IDENTITY	Identificador del mensaje.	BIGINT: es la tabla de mayor volumen del sistema.
id_usuario_emisor	INTEGER	FK → usuario NOT NULL CHECK (id_usuario_emisor <> id_usuario_receptor)	Usuario que envía.	Mismo tipo que usuario.id_usuario.
id_usuario_receptor	INTEGER	FK → usuario NOT NULL CHECK (id_usuario_emisor <> id_usuario_receptor)	Usuario que recibe.	Mismo tipo que usuario.id_usuario.
contenido	VARCHAR(1000)	NOT NULL	Texto del mensaje.	VARCHAR(1000): acota la mensajería directa.
fecha_envio	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Fecha y hora de envío.	TIMESTAMP automático.
leido	BOOLEAN	NOT NULL DEFAULT FALSE	Si el receptor ya lo leyó.	BOOLEAN: estado binario.

Clave primaria: pk_mensaje (id_mensaje). Clave sustituta BIGINT.

5. Implementación de las relaciones entre tablas

Las 36 llaves foráneas del modelo se declaran con restricciones nombradas (`fk_<hija>_<padre>[_rol]`). En todas se declara **ON UPDATE CASCADE**: las claves son sustitutas e inmutables, pero se deja explícito para que cualquier renumeración se propague. Para **ON DELETE** se aplicaron tres criterios:

- **CASCADE** en tablas asociativas y en datos que dependen de su padre (adjuntos, comentarios, publicaciones propias): la fila hija no tiene sentido sin el padre.
- **RESTRICT** cuando eliminar el padre perdería información académica o dejaría algo sin responsable (programa con estudiantes, creador de un grupo, organizador de un evento).
- **SET NULL** cuando la referencia es opcional y la fila hija puede sobrevivir sin ella (asignatura de un grupo o mentoría, grupo de un evento).

#	Tabla hija	Campo(s) FK	Tabla padre	ON DELETE	ON UPDATE	Justificación
1	`estudiante`	`id_usuario`	`usuario` (id_usuario)	CASCADE	CASCADE	Subtipo 1:1. Es un dato que no tiene sentido sin el padre; si el padre se elimina, la fila hija se elimina con él.
2	`estudiante`	`id_programa_academico`	`programa_academico` (id_programa)	RESTRICT	CASCADE	Se impide eliminar el padre mientras existan filas hijas, para no perder información académica o de responsabilidad.
3	`docente`	`id_usuario`	`usuario` (id_usuario)	CASCADE	CASCADE	Subtipo 1:1. Es un dato que no tiene sentido sin el padre; si el padre se elimina, la fila hija se elimina con él.
4	`usuario_habilidad`	`id_usuario`	`usuario` (id_usuario)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
5	`usuario_habilidad`	`id_habilidad`	`habilidad` (id_habilidad)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
6	`usuario_interes`	`id_usuario`	`usuario` (id_usuario)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
7	`usuario_interes`	`id_interes`	`interes` (id_interes)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
8	`sigue`	`id_usuario_seguidor`	`usuario` (id_usuario)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
9	`sigue`	`id_usuario_seguido`	`usuario` (id_usuario)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
10	`programa_asignatura`	`id_programa`	`programa_academico` (id_programa)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
11	`programa_asignatura`	`id_asignatura`	`asignatura` (id_asignatura)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
12	`docente_asignatura`	`id_docente`	`docente` (id_usuario)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.

#	Tabla hija	Campo(s) FK	Tabla padre	ON DELETE	ON UPDATE	Justificación
13	docente_asignatura	id_asignatura	asignatura (id_asignatura)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
14	mentoria	id_usuario_mentor	estudiante (id_usuario)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
15	mentoria	id_usuario_aprendiz	estudiante (id_usuario)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
16	mentoria	id_asignatura	asignatura (id_asignatura)	SET NULL	CASCADE	La referencia es opcional; si el padre se elimina, la fila hija se conserva sin esa asociación.
17	grupo	id_usuario_creador	usuario (id_usuario)	RESTRICT	CASCADE	No se puede eliminar al creador mientras el grupo exista: primero hay que transferir o eliminar el grupo.
18	grupo	id_asignatura	asignatura (id_asignatura)	SET NULL	CASCADE	La referencia es opcional; si el padre se elimina, la fila hija se conserva sin esa asociación.
19	usuario_grupo	id_usuario	usuario (id_usuario)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
20	usuario_grupo	id_grupo	grupo (id_grupo)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
21	evento	id_usuario_organizador	usuario (id_usuario)	RESTRICT	CASCADE	Un evento no puede quedar sin responsable: no se elimina al organizador mientras tenga eventos.
22	evento	id_grupo	grupo (id_grupo)	SET NULL	CASCADE	Si el grupo se elimina, el evento sigue vigente con su organizador, como evento individual.
23	usuario_evento	id_usuario	usuario (id_usuario)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
24	usuario_evento	id_evento	evento (id_evento)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
25	publicacion	id_usuario_autor	usuario (id_usuario)	CASCADE	CASCADE	Si se elimina la cuenta, se eliminan sus publicaciones.
26	publicacion	id_grupo	grupo (id_grupo)	CASCADE	CASCADE	Las publicaciones de un grupo se eliminan con él; SET NULL las haría visibles en el muro general, lo cual sería indebido.
27	publicacion_etiqueta	id_publicacion	publicacion (id_publicacion)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
28	publicacion_etiqueta	id_etiqueta	etiqueta (id_etiqueta)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
29	adjunto	id_publicacion	publicacion (id_publicacion)	CASCADE	CASCADE	Entidad débil: no existe sin su publicación. Es un dato que no tiene sentido sin el padre; si el padre se elimina, la fila hija se elimina con él.

#	Tabla hija	Campo(s) FK	Tabla padre	ON DELETE	ON UPDATE	Justificación
30	comentario	id_publicacion	publicacion (id_publicacion)	CASCADE	CASCADE	Entidad débil: no existe sin su publicación. Es un dato que no tiene sentido sin el padre; si el padre se elimina, la fila hija se elimina con él.
31	comentario	id_usuario_autor	usuario (id_usuario)	CASCADE	CASCADE	Si se elimina la cuenta, se eliminan sus comentarios.
32	comentario	id_comentario_padre	comentario (id_comentario)	CASCADE	CASCADE	Una respuesta no tiene sentido sin el comentario al que responde.
33	reaccion	id_usuario	usuario (id_usuario)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
34	reaccion	id_publicacion	publicacion (id_publicacion)	CASCADE	CASCADE	Fila de una tabla asociativa: si desaparece cualquiera de los dos extremos, la relación deja de existir.
35	mensaje	id_usuario_emisor	usuario (id_usuario)	CASCADE	CASCADE	Si se elimina la cuenta, se eliminan sus mensajes.
36	mensaje	id_usuario_receptor	usuario (id_usuario)	CASCADE	CASCADE	Si se elimina la cuenta, se eliminan sus mensajes.

6. El script DDL: estructura y verificación

El script `ddl_red_social.sql` está documentado con comentarios y se organiza así:

- **Encabezado:** propósito, SGBD, convenciones de nomenclatura y orden del script.
- **Esquema:** `DROP SCHEMA IF EXISTS ... CASCADE` y `CREATE SCHEMA red_social`, de modo que el script puede ejecutarse varias veces desde cero.
- **24 sentencias CREATE TABLE**, ordenadas para que cada tabla padre exista antes que sus hijas (catálogos académicos, identidad y perfil, grupos y eventos, contenido social). Cada tabla declara columnas, PK, UNIQUE, FK y CHECK como restricciones nombradas.
- **24 índices** sobre llaves foráneas (`idx_<tabla>_<campo>`).
- **COMMENT ON TABLE** para documentar cada tabla dentro del propio catálogo del SGBD.

Verificación

El script se ejecutó en PostgreSQL 16 sin errores. Se verificó que se crean 24 tablas, 36 llaves foráneas y 24 índices. Además, `pruebas_red_social.sql` inserta datos de ejemplo y comprueba que el motor rechaza los casos inválidos (correo repetido, autoseguimiento, mensaje a sí mismo, evento con fin anterior al inicio, tipo de publicación inválido, semestre fuera de rango, reacción duplicada, FK inexistente, campo obligatorio vacío, y los dos RESTRICT) y que las acciones CASCADE y SET NULL se comportan según lo definido.

Reglas que no se implementan con DDL declarativo

- **Disyunción de la especialización:** que un usuario no sea a la vez estudiante y docente no puede expresarse solo con PK/FK/CHECK; requeriría un disparador (trigger) o control en la aplicación.
- **Creador como administrador:** al crear un grupo, la aplicación debe insertar al creador en `usuario_grupo` con rol 'administrador'.
- **Mentor y aprendiz son estudiantes:** sí queda garantizado, porque ambas FK de `mentoria` apuntan a `estudiante`, no a `usuario`.

Cómo ejecutarlo

1) Abrir pgAdmin4, crear una base de datos (por ejemplo `pascualina`). 2) Abrir la herramienta Query Tool sobre esa base. 3) Cargar y ejecutar `ddl_red_social.sql`. 4) Opcionalmente ejecutar `pruebas_red_social.sql`. También por consola: `psql -d pascualina -f ddl_red_social.sql`.

7. Conclusiones

Jonathan Andrés Serna Velasco

Del modelo lógico al físico. Este trabajo me mostró que pasar del modelo lógico al físico no es solo copiar tablas a un SGBD: cada campo exige decidir un tipo, un tamaño y una restricción. Elegí PostgreSQL por su rigor con la integridad referencial y porque el material del curso está orientado a él. Decisiones como usar INTEGER en la mayoría de las claves y BIGINT solo en publicacion, comentario, adjunto y mensaje, o VARCHAR(500) en los comentarios frente a TEXT en las

publicaciones, me obligaron a pensar en el volumen y el uso real de cada dato.

Dificultades y dudas. Lo más difícil fue definir las acciones referenciales. Para cada una de las 36 llaves foráneas tuve que preguntarme qué debía pasar con la fila hija: CASCADE si no tiene sentido sin el padre, RESTRICT si se perdería información o quedaría algo sin responsable (como el creador de un grupo), y SET NULL si la referencia es opcional. Tuve dudas sobre si eliminar una cuenta debía borrar sus publicaciones, y sobre si los valores de campos como tipo_publicacion debían ir en una tabla aparte o en un CHECK; opté por el CHECK para mantener fiel el modelo lógico. También dudé de la nomenclatura, porque debía alinearla con las reglas del libro del curso.

Aspectos relevantes. Al revisar mi modelo de la Tarea 2 encontré que la clave primaria de docente_asignatura no incluía el periodo académico, así que un docente no habría podido orientar la misma materia en dos periodos; la corregí y la documenté. Escribir un script de pruebas me hizo ver el valor de verificar que las restricciones realmente rechacen los datos inválidos y que los borrados en cascada se comporten como se definió. Además comprendí los límites del DDL declarativo: la disyunción entre estudiante y docente no se puede garantizar solo con claves y CHECK, y requeriría un disparador o control en la aplicación. En conjunto, me llevo una comprensión más sólida de cómo un buen diseño físico protege la integridad de los datos desde la propia base de datos.

Anexo. Diagrama del modelo físico

Diagrama generado a partir de la misma definición que produce el script DDL: 24 tablas, claves primarias (PK), foráneas (FK), cardinalidad N:1 y acción ON DELETE sobre cada relación. Los colores agrupan las tablas por área temática.

